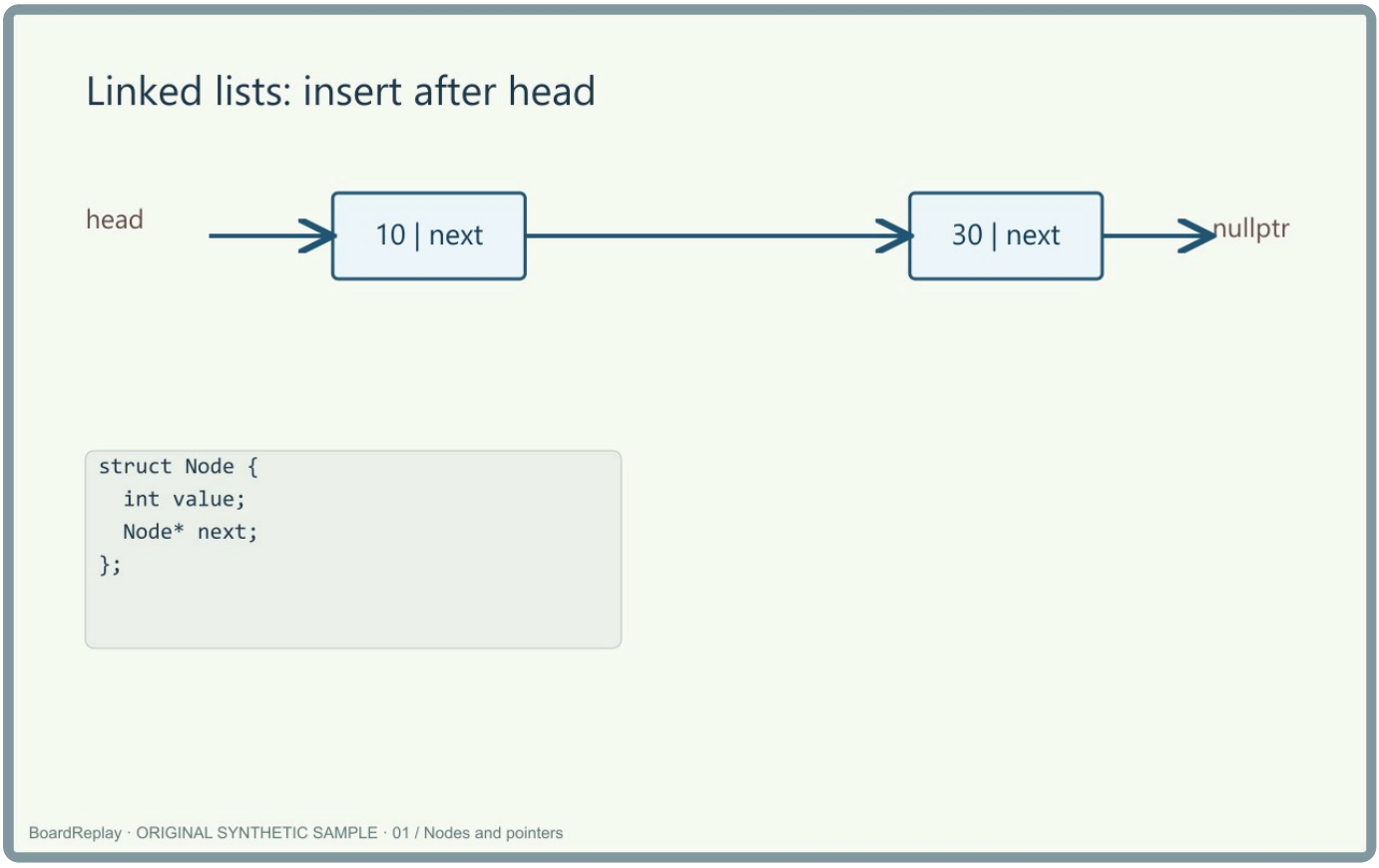
Linked lists: preserve the pointer, then insert

Data Structures and Algorithms · Synthetic sample
5 photographs

> Synthetic sample · Original sample photographs and author-written explanations. No live AI output.

PHOTO 01



sample-board-1.png

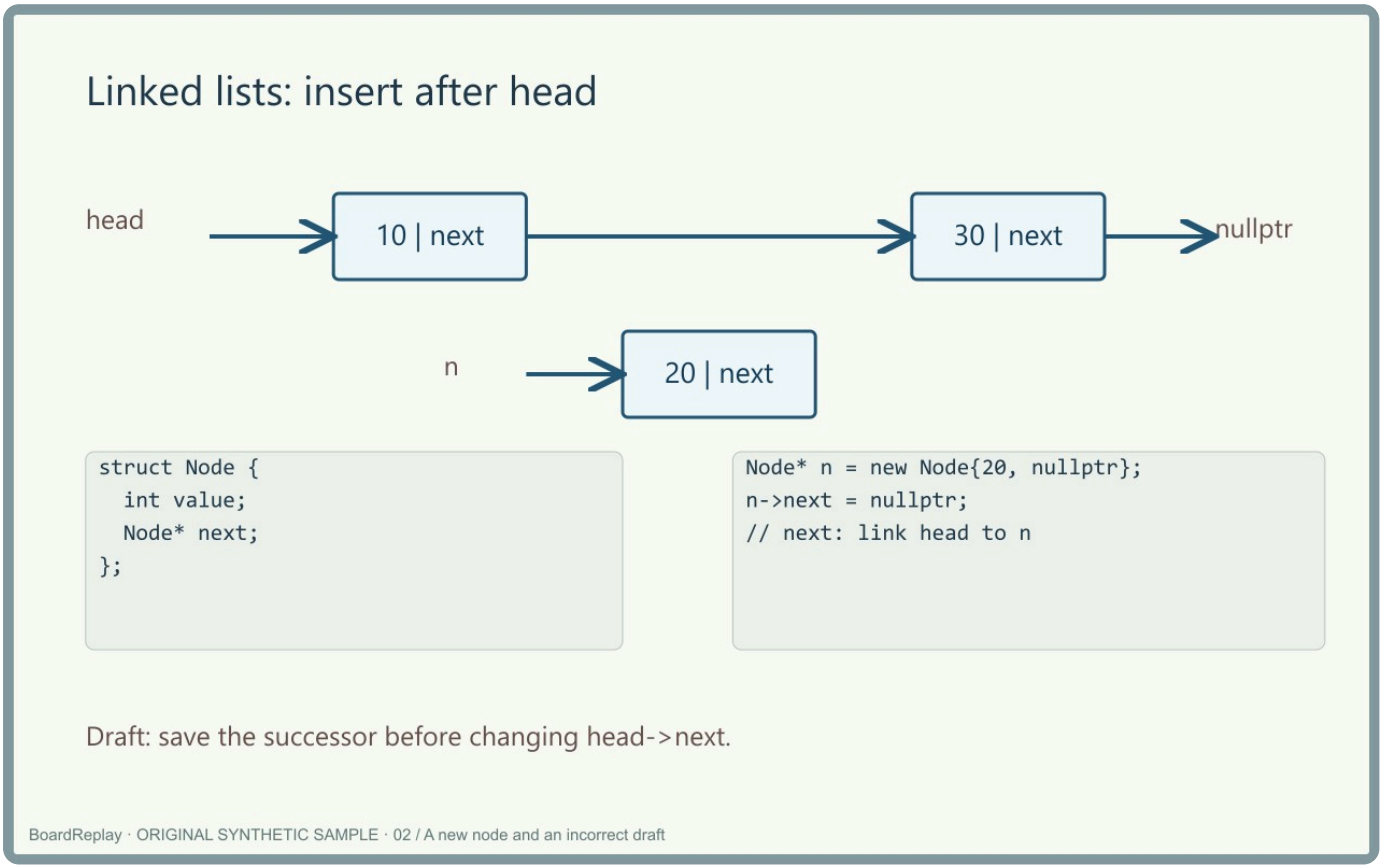
The first board shows head pointing to node 10, node 10 pointing to node 30, and the Node

The board introduces a singly linked list: each node stores a value and a pointer to the next node. The pointer `head` identifies the first node; `nullptr` marks the end. The C++ structure below the diagram gives the fields represented by each box. This establishes the state used by the following insertion examples.

```
struct Node {
  int value;
```

```
  Node* next;
};
```

PHOTO 02



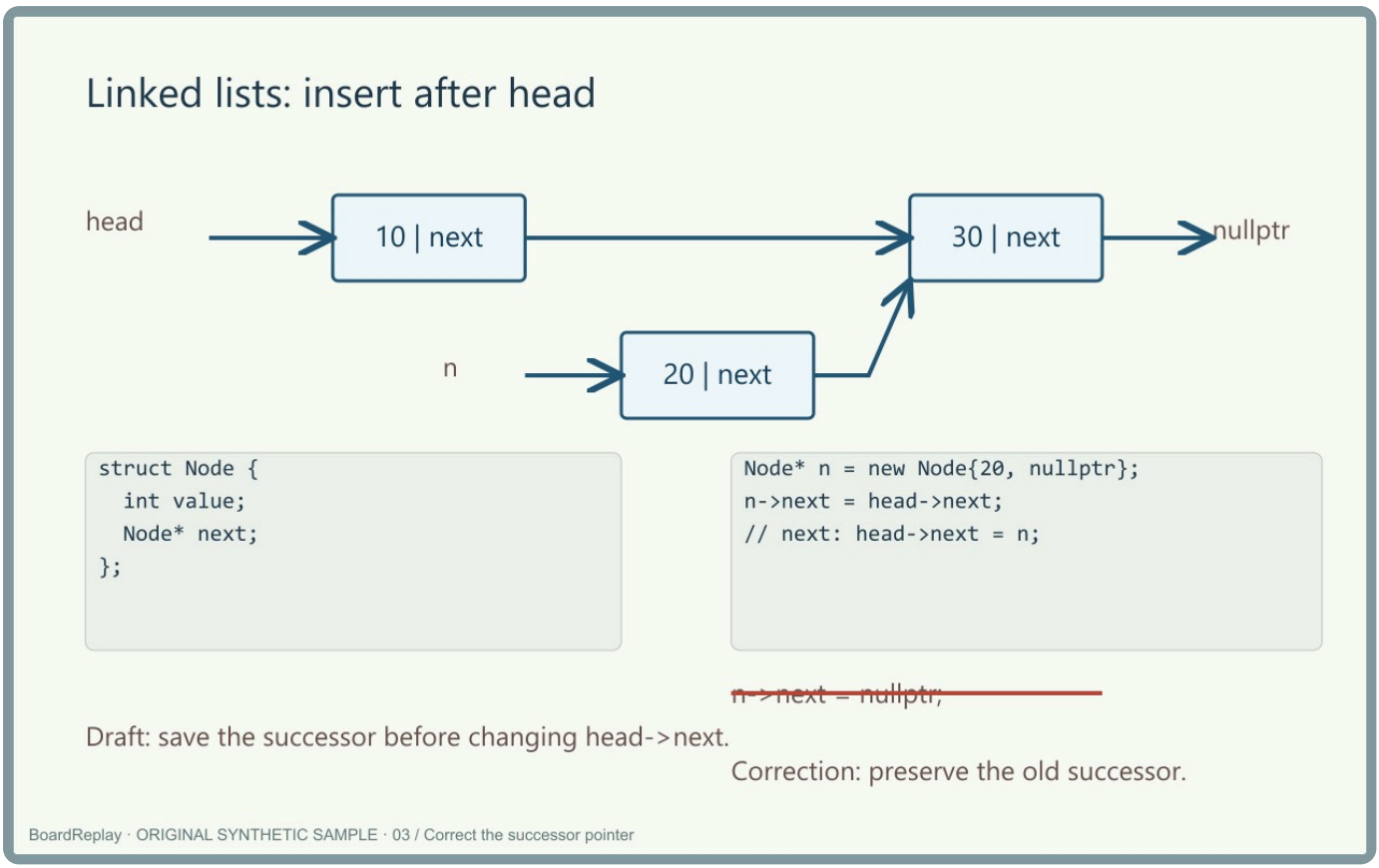
sample-board-2.png

A node containing 20 and an insertion draft appear. The observed draft says n->next = null

A new node is being inserted between two existing nodes. Preserve the successor before changing the predecessor: the new node first points to `current`, then the preceding link can point to the new node. The two assignments change different links. Reversing them can lose access to the remaining list. The original photographed code is the source to check for exact variable names.

```
Node* n = new Node{20, nullptr};
n->next = nullptr;
// next: link head to n
```

PHOTO 03



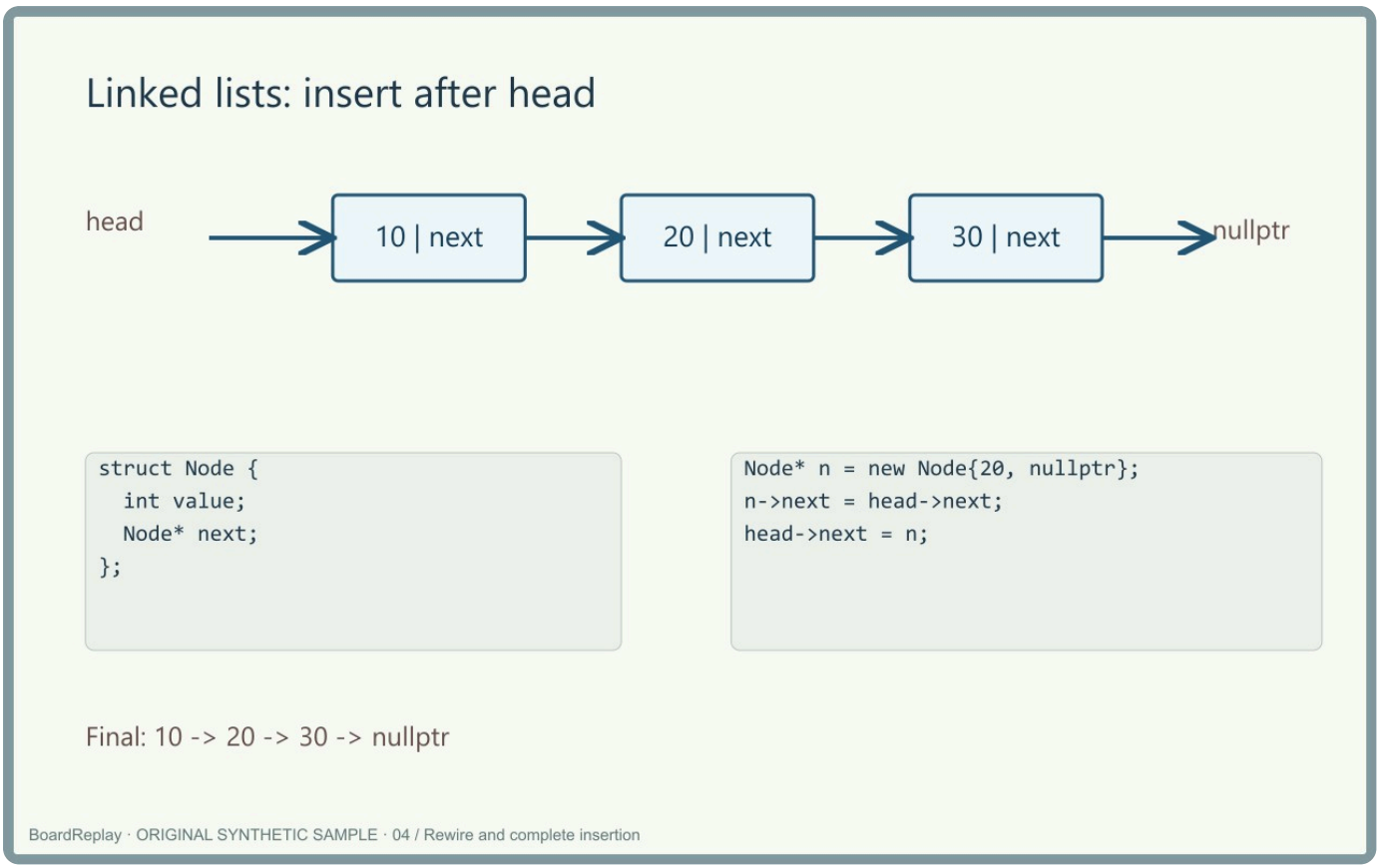
sample-board-3.png

The draft is visibly crossed out and corrected to n->next = head->next. Node 20 now points

This photo corrects the earlier insertion draft. Compare the crossed-out line and its replacement in the photograph: the new node's `next` pointer must preserve the old successor before the incoming link is changed. The correction matters because pointer updates are sequential operations, even when the resulting diagram looks simple.

```
Node* n = new Node{20, nullptr};
n->next = head->next;
// next: head->next = n;
```

PHOTO 04



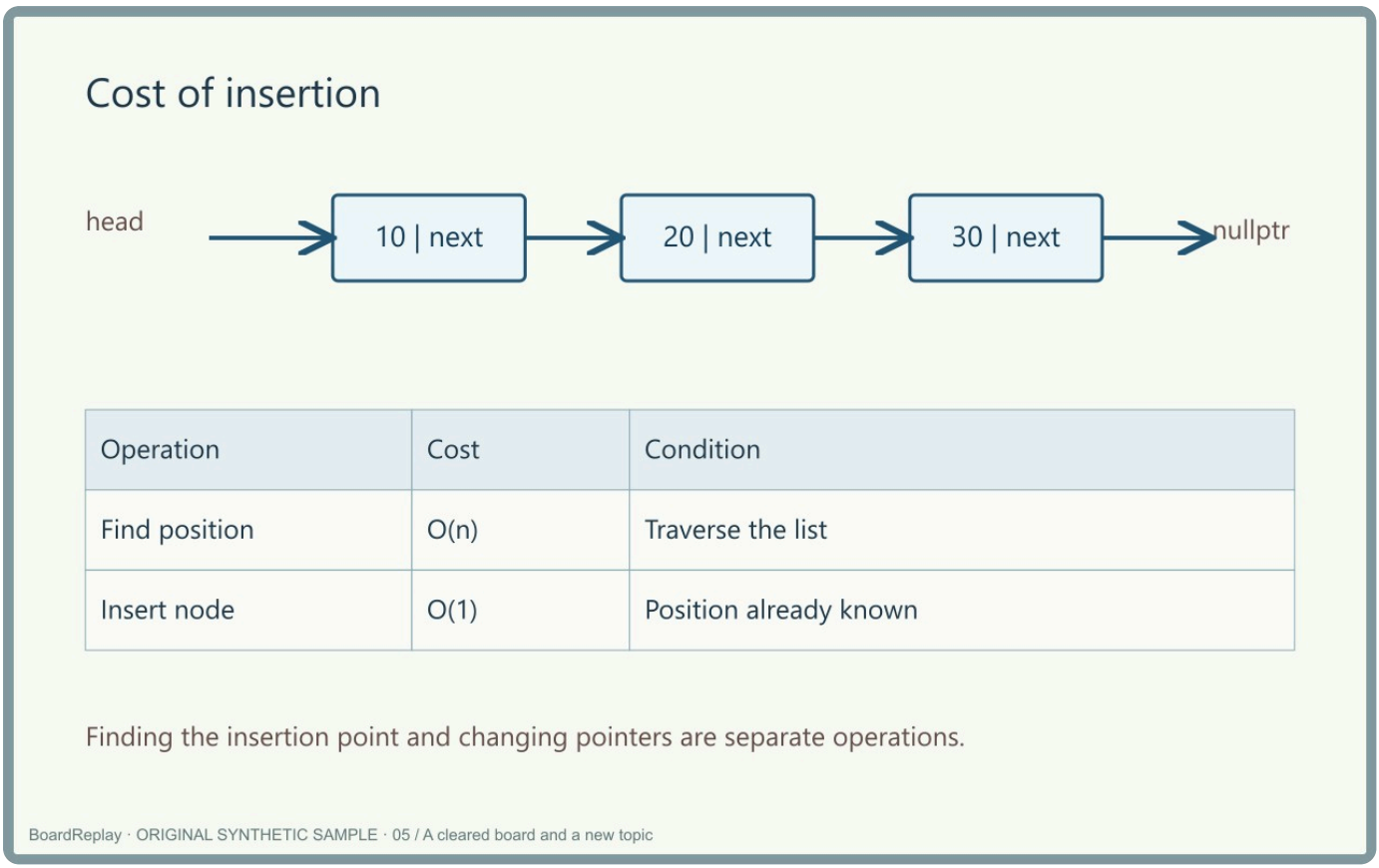
sample-board-4.png

The old 10-to-30 arrow and temporary annotations are erased. The code adds head->next = n,

The insertion has completed. Read the arrows from `head` to verify that every existing node is still reachable and that the inserted node occurs in the intended position. This photograph continues the same example; the unchanged node structure and pointer terminology need not be introduced again.

```
Node* n = new Node{20, nullptr};
n->next = head->next;
head->next = n;
```

PHOTO 05



sample-board-5.png

The board is cleared for the insertion-cost topic. The final list is redrawn above a table

The discussion moves to insertion cost. Finding a position can require traversing the list, giving $O(n)$ work. Once the insertion position and relevant link are already known, changing the links requires $O(1)$ work. The table distinguishes search cost from the pointer update itself.

BoardReplay · Original photographs with derived explanations. Review uncertain content against the sources.